

Recitation 4: Trees and Lists

For the questions on trees, assume each node is a data structure like the following:

```
class Node:
    val: Int
    left: Null // or Node
    right: Null // or Node
```

For linked lists, you may assume a node type

```
class ListNode:
    val: Int
    next: Null // or ListNode
```

1. In class you saw an iterative algorithm for depth-first search. Design a *recursive* algorithm for depth-first search (you may assume preorder: visit the node, then its left subtree, then its right subtree).
2. A level-order traversal of a tree visits each node of a given level before descending to the next level. Write an algorithm to print all the nodes of a tree in level-order. (Hint: use a queue.)
3. Write an algorithm that returns the height of a tree (max root-to-leaf path length). Assume the height of an empty tree is 0, and the height of a single-node tree is 1.
4. Given a sorted list of distinct integers $a_1 < a_2 < \dots < a_n$ (for example, the result of an inorder traversal of some binary search tree), write an algorithm to construct a *height-balanced* binary search tree containing exactly these keys.
5. Given two sorted linked lists L_1 and L_2 (in nondecreasing order), merge them to produce a single sorted linked list containing all the elements of L_1 and L_2 .
6. Given an array $A[0 \dots n-1]$ representing a complete binary tree, write an algorithm to check in $O(n)$ time whether it satisfies the *min-heap* property (every node is $\leq$ its children). (For a max-heap, reverse the inequality.)